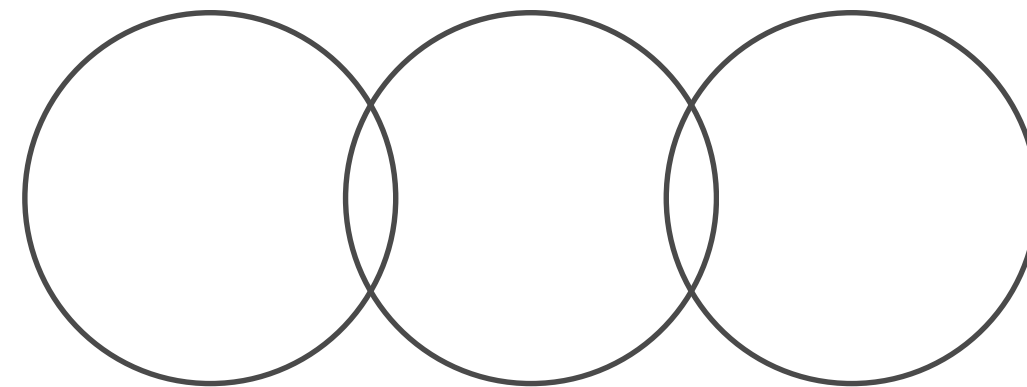
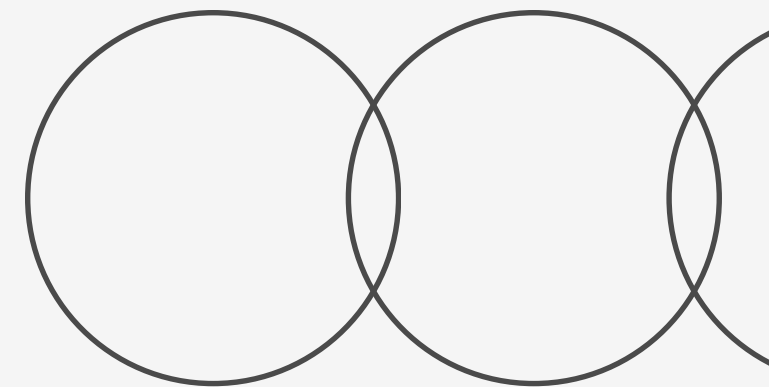


Used Car Price Prediction System



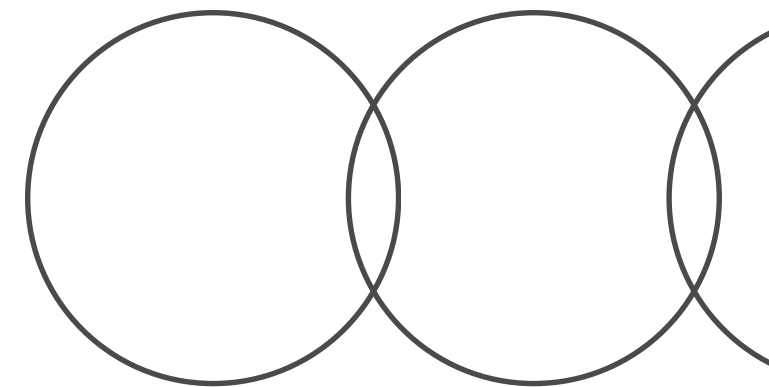
The problem

- The used-car market is unpredictable
- Prices depend on many factors (mileage, age, fuel type, market trends)
- Buyers cannot estimate depreciation accurately
- Sellers have difficulty pricing fairly
- No widely accessible interactive tool exists



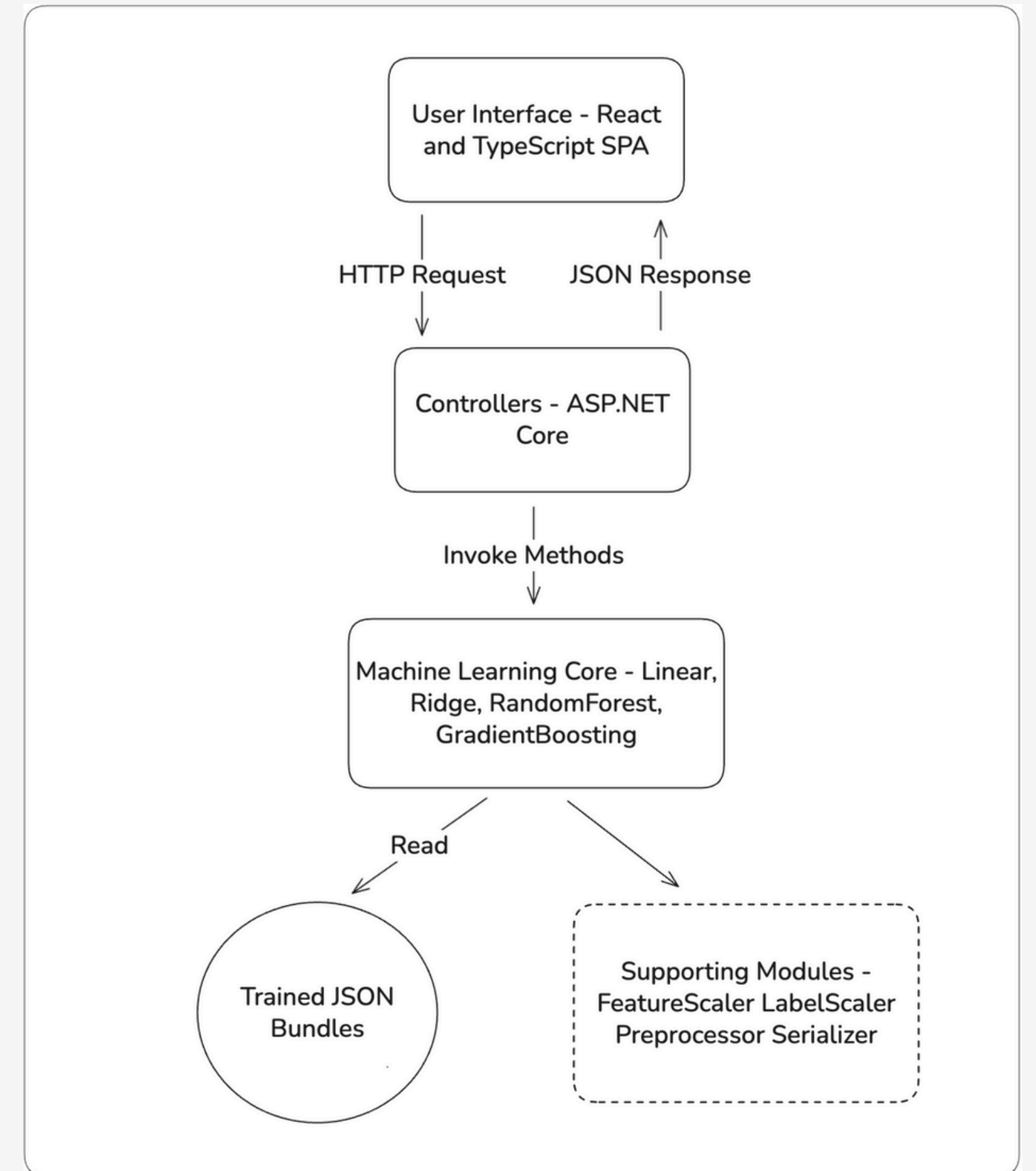
How does the project solve the problem

- Uses real historical data to learn pricing patterns
- Predicts fair market prices for any car configuration
- Provides multi-year depreciation forecasts
- Allows side-by-side comparison of two cars
- Uses ensemble ML models for high predictive performance
- Offers a fast and easy-to-use web interface



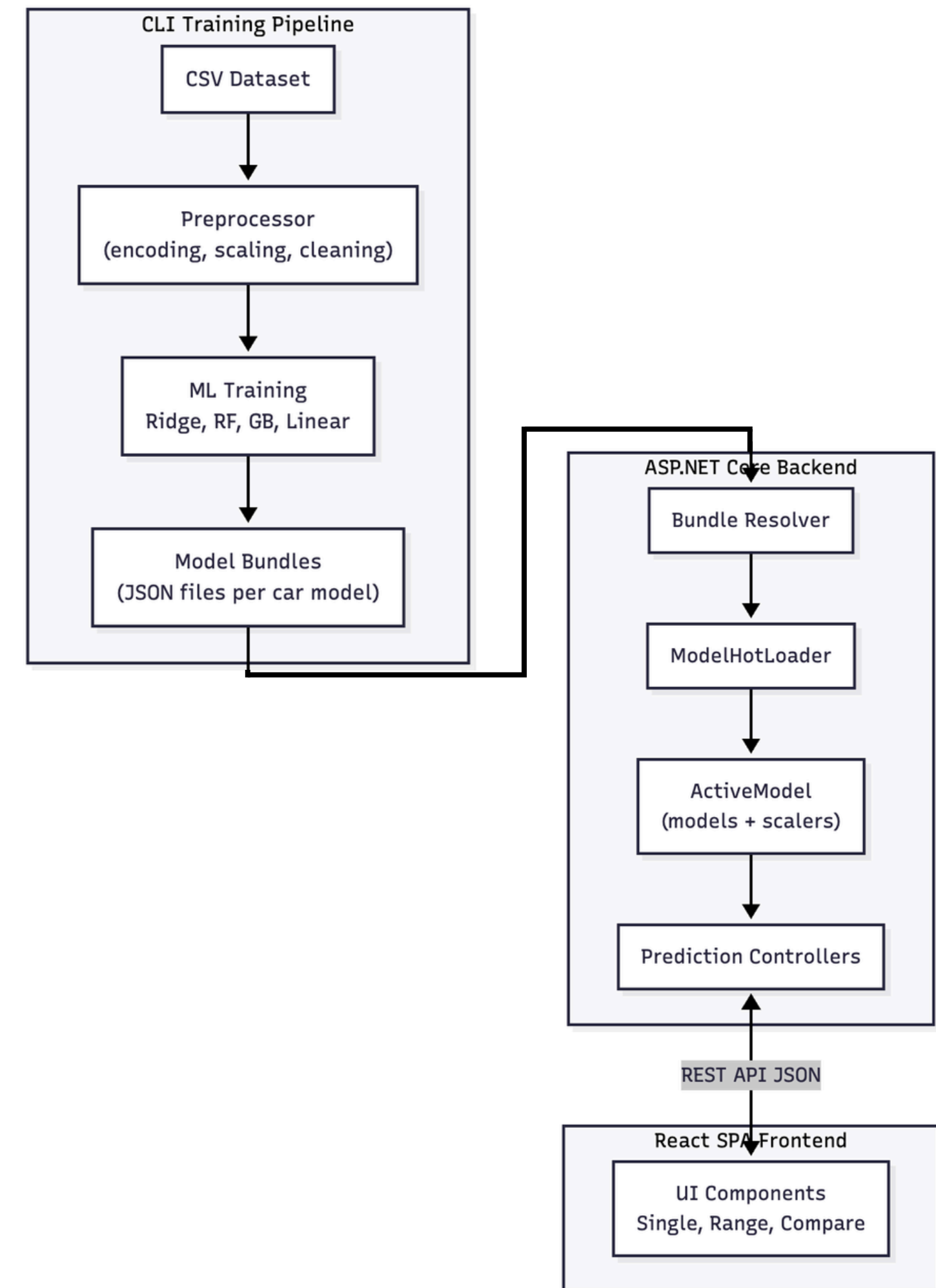
Project overview

- ASP.NET Core backend
- React + TypeScript frontend
- CLI trainer for model building
- Custom ML algorithms
- Trained model bundles stored as JSON
- 4 types of predictions (single, comparison, range(single, comparison))



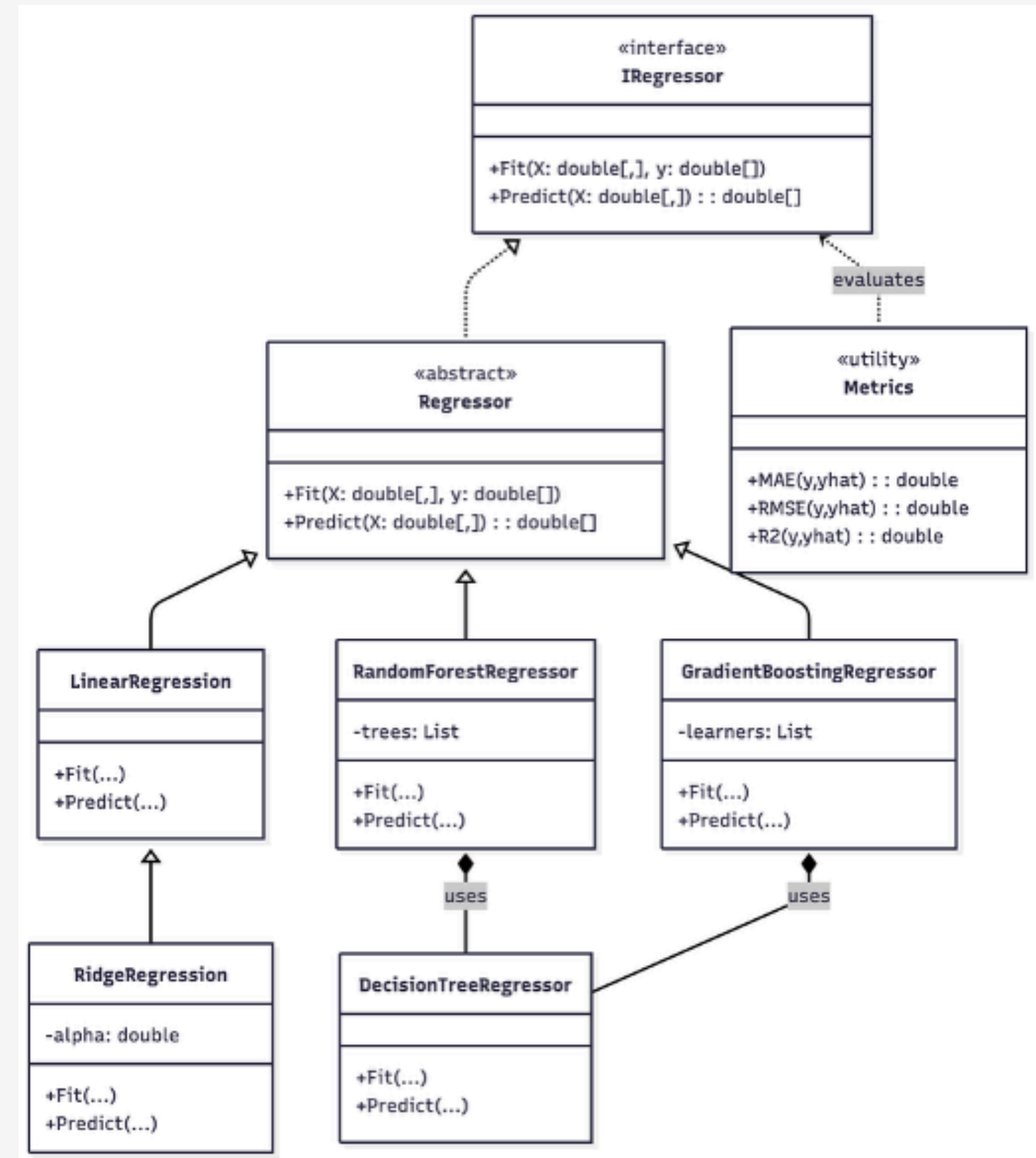
Architecture Diagram

- CSV Dataset → CLI Trainer → Model Bundles
- ASP.NET Core API (PredictionController, ModelHotLoader, ActiveModel)
- React SPA UI (Single prediction, Range prediction, Compare)



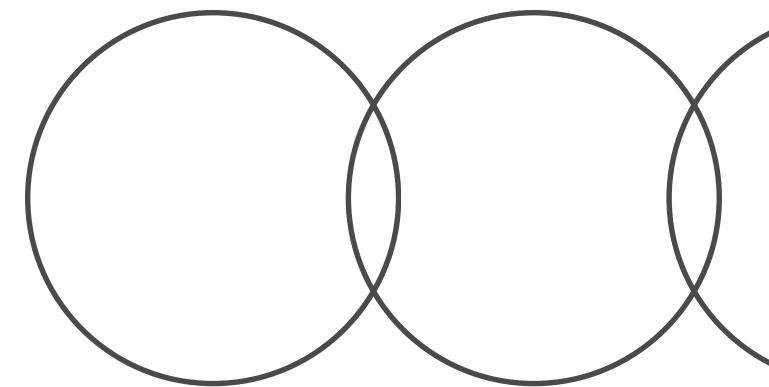
Class Hierarchy

- CSV Dataset → CLI Trainer → Model Bundles
- ASP.NET Core API (PredictionController, ModelHotLoader, ActiveModel)
- React SPA UI (Single prediction, Range prediction, Compare)



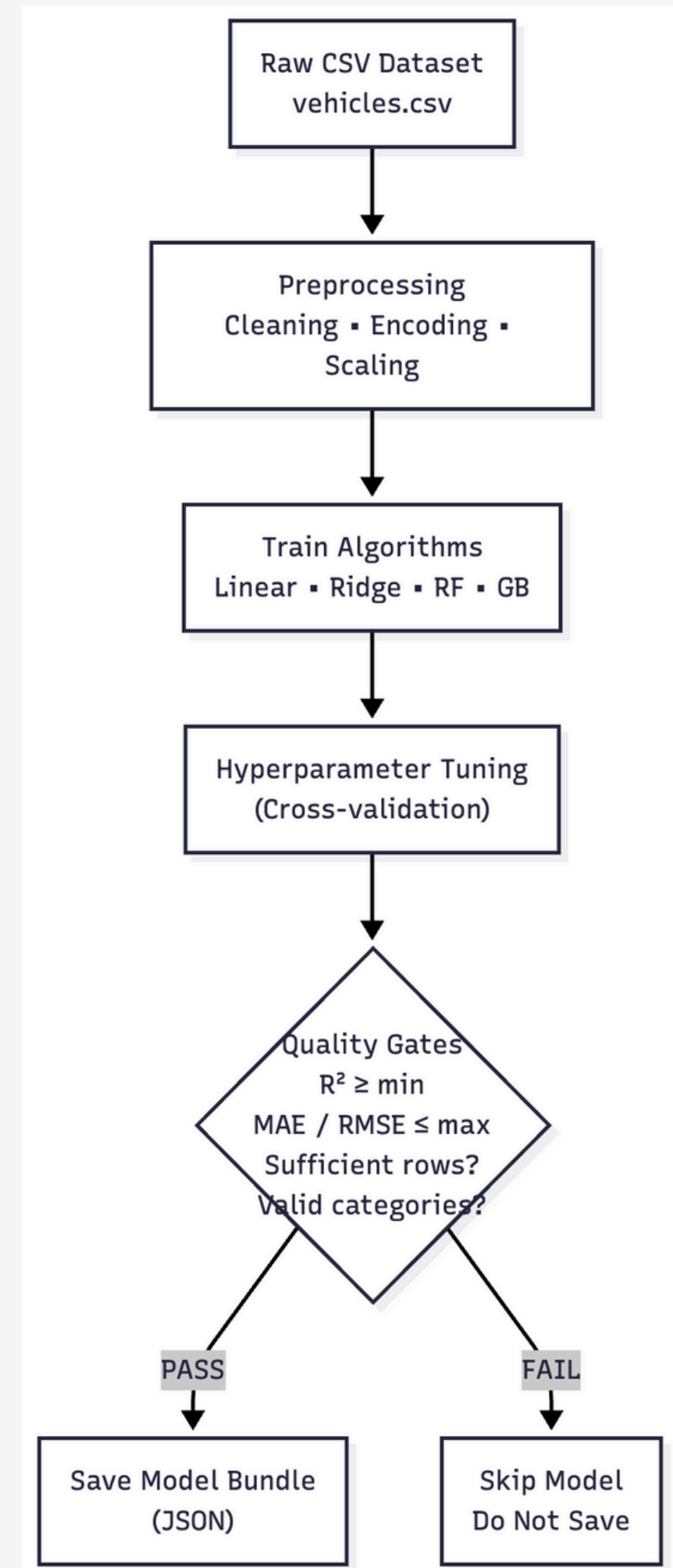
Algorithms used

- Linear Regression – provides the simplest baseline for price estimation
- Ridge Regression – adds stability and prevents overfitting in trend modeling
- Random Forest – captures nonlinear patterns that linear models miss
- Gradient Boosting – delivers highest accuracy by learning from residual errors
- Hybrid Models – combine Ridge + RF/GB for stronger, more reliable predictions



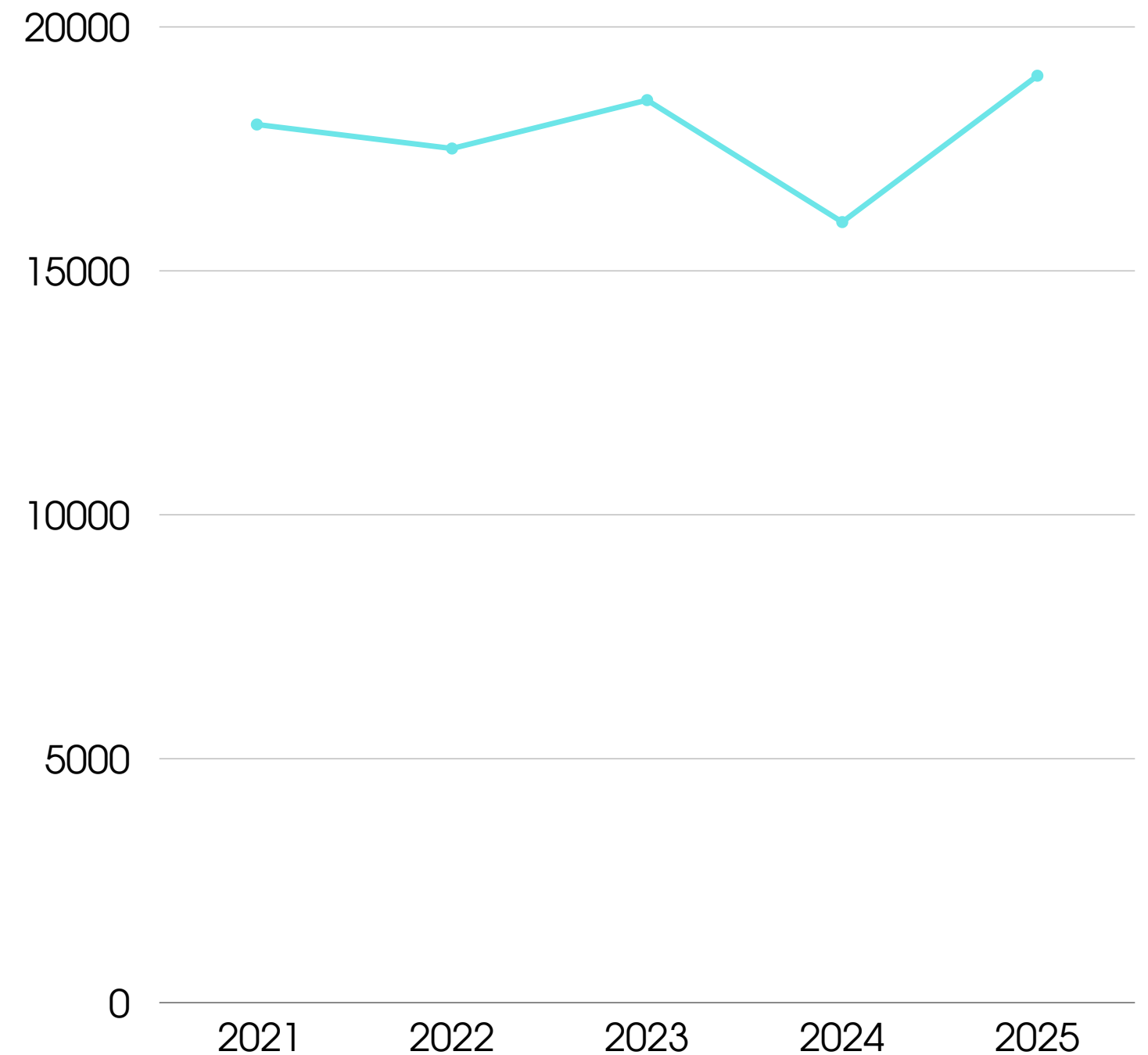
Training Pipeline & Quality Gates

- Processes raw vehicle data (cleaning, encoding, feature scaling)
- Trains four algorithms on each (make, model) pair
- Tunes hyperparameters using cross-validation
- Applies strict quality gates (R^2 , MAE, RMSE thresholds)
- Skips models with insufficient data or missing categories
- Saves only high-quality models as bundles



Challenges

- RF/GB models were unstable for long-term forecasts → required using Ridge for range predictions
- Raw data had inconsistent model names → needed a custom canonical ID system
- Ensuring safe model hot-loading



Results & Findings

- High model accuracy across most car models (strong MAE/RMSE and R^2 scores)
- RF/GB + Ridge Regression models performed best overall
- Strict quality gates prevented weak models from being saved
- Backend test coverage: 84%
- Frontend test coverage: 98%

Linear Regression

Metrics:

MSE:	8090455.35
MAE:	1891.52
R^2 :	0.750

Timing:

Total Time:	1.385 s
-------------	---------

Ridge Regression

Metrics:

MSE:	8104326.59
MAE:	1895.39
R^2 :	0.750

Timing:

Total Time:	0.198 s
Trials:	200
Mean Trial:	0.001 s

Random Forest

Metrics:

MSE:	4473092.67
MAE:	1334.63
R^2 :	0.860

Timing:

Total Time:	43.774 s
Trials:	60
Mean Trial:	0.730 s

Gradient Boosting

Metrics:

MSE:	3924617.62
MAE:	1226.02
R^2 :	0.880

Timing:

Total Time:	410.008 s
Trials:	60
Mean Trial:	6.833 s